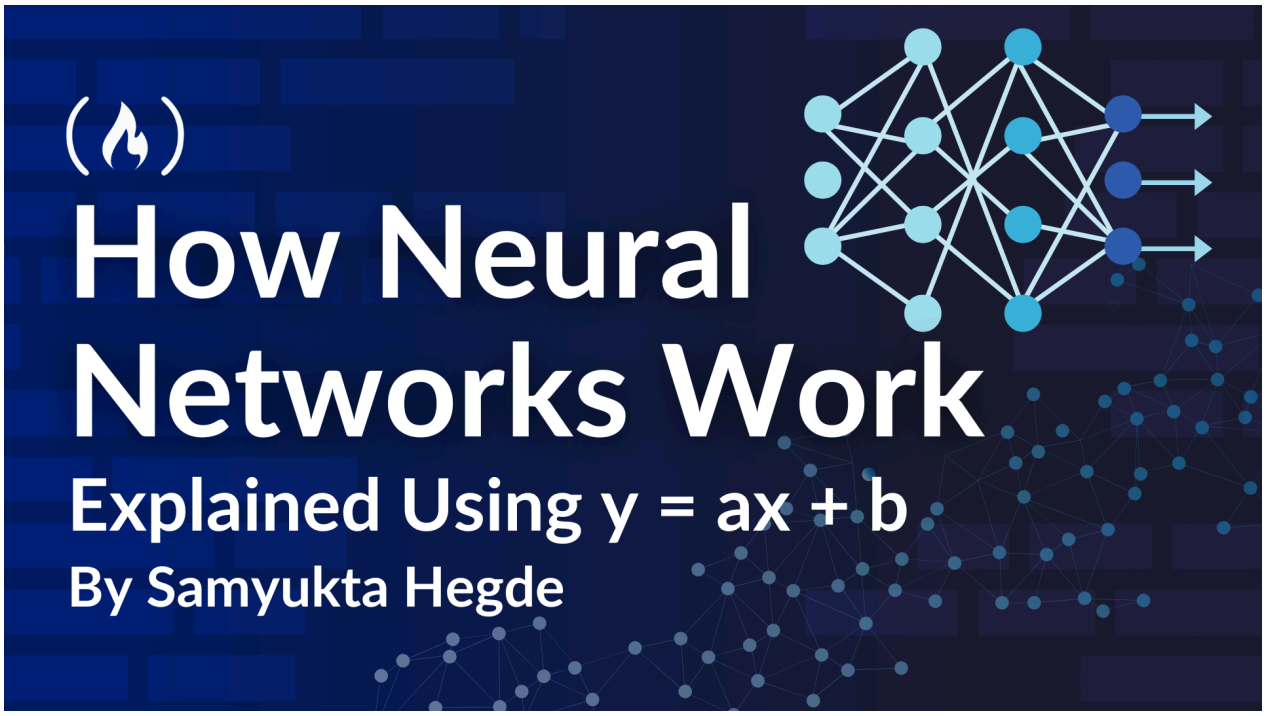
JANUARY 8, 2026 / #DEEP LEARNING

How Neural Networks Work – Explained Using the Straight Line Equation y = ax + b

Samyukta Hegde



Did you know that every data scientist who builds a complex neural network starts with a fundamental question, “How does the output change when the input changes?“

A straight line equation `y = ax+b` answers it in the simplest way possible. `y` can increase, decrease, or stay the same when `x` changes.

On the other hand, a deep neural network tries to answer it in a flexible way. It's only possible because of multiple layers of straight line calculations stacked one over another along with non linear adjustments to help the network adapt and produce the desired result.

Since a straight line is the essence of neural networks, I think it's time we try to understand the subtle details of `y = ax+b`, which I refer to as the **magical equation**. We'll also go through the basics of linear regression and classification, which should help you understand the progression of a simple straight line to a complex deep neural network.

Table of Contents

- Prerequisites
- y=ax+b
- Linear Regression
- Linear Classification
- Comparison
- Key Additions to Help Build Deep Neural Networks
- Modelling a Deep Neural Network
- Final Thoughts

Prerequisites

- A basic understanding of linear algebra, particularly `y=ax+b`.
- General idea about linear regression and classification.

- Familiarity with the concept of deep neural networks.

y=ax+b

A straight line simply means that output changes steadily as input changes. There are no surprises (that is, no non linearity). Let's analyze it properly.

```
y => Output variable
x => Input variable
a => Amount by which y changes when x changes (slope)
b => Value of y when x is 0 (y intercept)
```

We can take an example and model it in the same form to understand it better.

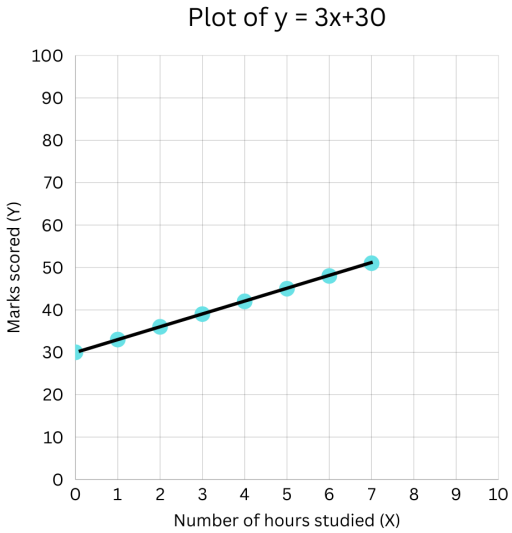
Ms. Poly is a math teacher who wants to formulate a study plan for her students to excel in an upcoming final exam. For simplicity, she creates a rule of thumb using only one factor: the number of hours studied per week. It has a direct impact on the marks scored by a student.

Before beginning, she makes certain assumptions:

- Every student is capable of scoring at least 30 without studying.
- For every hour a student studies, an additional 3 marks can be scored.

She then comes up with the following equation based on her ideas: `y = 3x+30`

```
y => Marks scored.
x => Number of hours studied.
a=3 => Increase in marks for every hour studied
b=30 => Minimum marks
```



In the above graph, she plots the points based on the results of the equation. As expected, it is a straight line. If she needs the marks scored for `9` hours of study, she can get it by just substituting `x=9` in `y=3x+30`. Note that the data (`x` and `y`) are just based on her hunch and aren't real.

But Ms. Poly wants to guide her students on how to prepare for the final exam based on actual data. So she conducts a pop quiz and grades it. In order to formulate a study plan, she interviews her students and collects information on how many hours they study math per week. She creates a table with two columns: number of hours studied (`x`) per week and marks scored (`y`). She tries her old formula `y=3x+30`, but it doesn't seem to work. Thus, she doesn't have any sensible equation describing the relation between `x` and `y`.

Let's assume that a new student who hasn't attended any exam (no `y` available) joins the class the next day, and Ms. Poly only knows the number of hours dedicated per week (`x`). How can she answer the question below?

If the new student studies for a certain number of hours (`x`), what can be the marks scored (`y`) in the exam?

It's impossible unless there's an equation defining the sample data. So, her task is to find one that fits the given points. This process is called curve fitting or regression.

Linear Regression

The core idea of linear regression to find a straight line that captures the trend of the existing data to facilitate predictions for new input data. Now, let's dive straight into the example to understand the concept better.

Ms. Poly is determined to arrive at a solution. She plots the collected data on a graph to get a better picture.

She has absolutely no idea how `x` and `y` are related. So, she must figure out a formula, by trial and error, that roughly fits the points. She has to start with an intuitive guess, try to improve it in the subsequent steps and then arrive at the best possible solution.

Trial 1: Ms. Poly begins with her previous straight line equation.

```
y = 3x+30
```

She substitutes different values of `x` and plots it alongside the collected input data. This way she can get a clear picture of the differences in her assumption and reality.

Trial 2: She observes that the line needs a little more slope. This simply means that, in reality, more marks are being scored for every additional hour of study. By changing it from `3` to `4`, the equation becomes:

```
y = 4x+30
```

The following graph depicts the new line alongside the sample data:

Trial 3: It looks better but she feels there is a need to shift the whole line upwards. This means that higher marks are being scored even if a student doesn't dedicate any time for math in a week. She decides to retain the previous slope but changes the starting marks by `10`, thus arriving at:

```
y = 4x+40
```

This particular line covers most of the points and can be considered the best possible solution.

Now, if she wishes to ascertain the marks scored by the new student who studied for `3.5` hours, she pins the value inside the formula and calculates the answer: `y = 4*(3.5)+40=54`

We saw how Ms. Poly arrived at a straight line equation to predict the output for an unknown input. Now she can chalk out a study plan for her class based on the equation.

Here, an expression is formulated to ascertain the change in output when the input changes. It looks like Ms. Poly is thinking like a data scientist. She has in fact modelled a very simple neural network for regression. The equation `y=4x+40` can be considered as the only neuron (processing unit) within it. She's adjusted the parameters `a` (weight) and `b` (bias) to arrive at the final formula which covers most of the points (thus minimizing the loss).

Here's a breakdown of the `y = 4x+40` equation:

```
y => Marks scored.
x => Number of hours studied.
a=4 => Increase in marks for every hour studied
b=40 => Minimum marks
```

At present, it is a rudimentary neural network which has no layering and non-linearity.

Now let's shift our attention to a completely different scenario. Ms. Poly, being a teacher, wants to ensure that all her students pass the exam. Assuming, as an end result, she's not interested in predicting the marks scored. She just wants to know:

If a student studies for a certain number of hours (`x`), will the student pass/fail(y) the exam?

This leads her to the process of classification.

Linear Classification

The linear classification process uses a simple straight line to divide the data into categories or classes. The line acts as a boundary so that the classes fall on either side of it. First, Ms. Poly defines the boundary condition for pass and fail.

If marks scored>=50, pass

If marks scored<50, fail

According to the data table, `x=3` corresponds to `y=52` (boundary condition). Therefore she considers `x=3` as the classification line******.

`x=3` seems to segregate the points into the categories properly. She tries to confirm it by substituting another value. Thus, if a student studied for `9` hours, the score would lie towards the right side of `x=3`. So, they'd pass as per the classification equation.

Again, she's arrived at an expression to ascertain the change in output when the input changes. But here, she has modelled a basic neural network for classification. The equation x=3 is the only neuron within it. It can be considered to be having two parts as explained below.

1. **Pre-Activation Part:** This portion of the neuron computes an intermediate value which is helpful in further processing. She's figured out the parameters `a` (weight) and `b` (bias) to arrive at the following formula: `z = x-3`

   ```
   z => Intermediate Value.
   x => Number of hours studied.
   a=1 => Influence of the number of hours studied on the marks scored
   b=-3 => Minimum number of hours to study to pass the exam = 3
   ```

2. **Activation Part:** This portion triggers the neuron to make decisions based on a threshold value. The following equation segregates the points into two classes.

```
y = 1 (Pass) if z>=0
y = 0 (Fail) if z<0
```

This is a very plain neural network which has no layering and non-linearity but has pre-activation and activation parts inside a neuron.

Comparison

We looked at the examples of both linear regression and classification used by Ms. Poly. **Regression** helps in predicting a value while **Classification** helps in decision making. Let's draw a small table to summarize the differences.

Upon careful observation we notice that both answer the question of how input change affects output.

But at a slightly higher level of complexity than a straight line. Because in the case of both regression and classification, we try to figure out the equation parameters by trial and error.

Here, since the requirements are simple, Ms. Poly just uses a straight line to solve both. A simple linear equation can handle only one steady trend. But in real life, problems that need solving are far more challenging and unpredictable. Some examples are:

Image Classification: An output label is produced based on the input images.

Text Translation: An English sentence can be given as an input to be translated to say, Spanish.

Chatbots: A text prompt is typed in by a user and a meaningful and relevant output is generated.

She probably should have to use a deep neural network if both data and task were complex. That presents another question: **How does one build a deep neural network?**

We will explore it further by extending the same example to a more realistic version.

Key Additions to Help Build Deep Neural Networks

In the above sections, we noted that Ms. Poly was interested in predicting the exam results of a student using just one factor - number of hours studied. However, in practice, is that one factor sufficient in determining the marks scored or whether the student passes the exam?

No. It's not enough. She needs to take into account a lot of aspects like:

- Number of hours studied
- Number of hours of sleep/rest
- Burnout due to over-studying
- Difficulty level of topics in math
- Pattern of the exam, and so on.

All the above neither act independently nor do they have a simple linear relation with the marks scored. So, she has to solve this problem by stacking the contributing factors one above the other in layers and also adding the element of non linearity. Let's take a look at each in detail.

Layering

Burnout leads to lower score whereas good sleep increases score. But burnout can be reduced if the student is well rested. So, the impact on the final score when these two factors interact should be taken into account. This is possible only when the system solves it in layers. The first layer can deal with how they independently influence the score, the next layer can explore the interaction between them.

Non-Linearity

If the number of hours studied increases, the score might increase but when burnout overpowers the effect of study hours, the score reduces. The combined effect results in a non-linear graph. There is a rise and then dip in the score based on number of hours studied. It's evident that the relationship is not straightforward as in a straight line. That's where it becomes necessary to add non-linearity in the calculations. It helps the system to respond differently according to the conditions, allowing for flexibility in dealing with real world data and conditions.

Thus, Ms. Poly would have to extend the idea of linear regression/classification by including layering and non-linearity to build a fully functional neural network to help build a practical study plan.

Modelling a Deep Neural Network

Ms. Poly should start the work on modelling a deep neural network by following the steps mentioned below:

Step #1 - Define the Problem Clearly

The following factors should be considered before she begins the process of modelling:

- What are the input features?
- What are the output features?
- What type of problem is it (regression/classification)?

Step #2 - Define the Input Layer

The input features form the first layer. There is no computation in this stage. They are represented as:

```
x1: Number of hours studied
x2: Number of hours of sleep/rest
x3: Burnout due to over-studying
x4: Difficulty level of topics in Maths
x5: Pattern of the exam
```

Step #3 - Define the First Hidden Layer

This step consists of two parts:

Apply Linear Transformation: The actual learning begins here. A straight line equation is used to understand the combined effect of the inputs. The general formula is `z=Wx+b`.

```
z: Intermediate value or Pre-activation
W: Weight matrix which consists of values corresponding to the impact of
each input feature
x: Matrix consisting of input features, [x1, x2, x3, x4, x5]
b: Bias which represents the initial assumptions of the teacher(when x=0)
```

It looks similar to a linear regression/classification equation. At first `W` and `b` are initialized to random values. Then in the subsequent steps, they are adjusted like it was done in earlier examples. We can consider the following combinations assuming we have two neurons in this layer:

Neuron 1: It can focus on study hours, burnout, and rest, with other features contributing less significantly.

Neuron 2: It can emphasize more on the difficulty level of the topic and the exam type compared to other inputs.

It's important to note that this layer doesn't calculate the interactions between the features but only on the way different linear combinations work together but independently. To make it clearer, how they contribute independently are added together. We don't know how one input feature influences the other. For example, we know sleep increases score and burnout reduces score, but what we don't know at this stage is if sleep reduces burnout, which in turn can influence the final score.

Add Non-Linearity: This step, also called activation, helps in capturing the complexities in different combinations of the features. Less study results in low marks, and too much burnout also results in low marks. It means there is a curve in the score graph which can't be represented by a linear equation. The activation function is applied to the intermediate value and can be expressed as:

a = g(z)

```
a: Activation output
g: Activation function
z: Intermediate value or Pre-activation
```

For example: `ReLU` is an activation function which outputs `z` only if `z` is positive, else `0`.

y = ReLU(z)=max(0,z)

We can see that it has no steady slope and is a non-linear activation function. It can suit this scenario as it lets the value pass through to the next layer only if the combined effect of features is greater than 0. Neuron 1 will let it's output go to the next layer only if the intermediate value (`z`) that results from study hours, burnout and rest, is large enough to be influencing the final decision, else it's ignored. There are multiple options for non-linear activation functions that one can choose from.

Step #4 - Stack Layers One Above the Other

This step helps in learning the mutual interactions between the inferences learned from the first hidden layer. The network attempts to understand the intricate details of the influencing factors and build a stable system. It is here that details of whether sleep reduces burnout are figured out. Every layer consists of linear and non linear transformations applied on the input, which are values obtained from the previous layer. Likewise multiple layers can be stacked one over the other based on the requirements. In this example, for representation, we have taken two hidden layers with two neurons each. The number of layers and neurons can vary based on requirements.

Step #5 - Define the Output Feature(s)

This appears to be the final stage in a deep neural network. Ms. Poly can decide what she wants for output: predict the marks scored by a student or predict if the student passes/fails the exam. If she wants the final marks scored, she just has to apply linear transformation in the neuron in the final layer to produce the output. If she wants pass/fail status, she has to apply both linear and non-linear transformations to achieve the desired results.

The diagram below shows an abstract representation of the deep neural network.

The next steps are:

Training the model: The network is trained in the following way:

- Random weights and biases are assigned to the linear transformation portions of the network.
- Then the network makes a prediction which is compared with the expected result.
- If there are gaps between the actual result and the predicted result, corrections are made in weights and biases (this step is similar to what was done in linear regression and classification).
- The steps above are repeated until the results improve.

Using the model: After the model has been trained, it is capable of yielding results for new input values.

Final Thoughts

In this article, we began with the basics of a straight line equation. Then we gradually navigated through slightly more elaborate concepts like linear regression and classification. They laid the groundwork for delving into the seemingly mysterious deep neural networks. But they are in fact built by stacking layers of linear transformations and non-linear activations, which help understand sophisticated real world patterns.

Despite all the complexities and layers, we can see that the straight line remains the foundation upon which neural networks are built. As we saw earlier, the equation that a deep neural network begins with is our *magical equation:* `y = ax+b`.

Samyukta Hegde

Read more posts.

If this article was helpful, share it.

Learn to code for free. freeCodeCamp's open source curriculum has helped more than 40,000 people get jobs as developers.

Get started

freeCodeCamp is a donor-supported tax-exempt 501(c)(3) charity organization (United States Federal Tax Identification Number: 82-0779546)

Our mission: to help people learn to code for free. We accomplish this by creating thousands of videos, articles, and interactive coding lessons - all freely available to the public.

Donations to freeCodeCamp go toward our education initiatives, and help pay for servers, services, and staff.

You can make a tax-deductible donation here.

Trending Books and Handbooks

REST APIs	Clean Code	TypeScript
JavaScript	AI Chatbots	Command Line
GraphQL APIs	CSS Transforms	Access Control
REST API Design	PHP	Java
Linux	React	CI/CD
Docker	Golang	Python
Node.js	Todo APIs	JavaScript Classes
Front-End Libraries	Express and Node.js	Python Code Examples
Clustering in Python	Software Architecture	Programming Fundamentals
Coding Career Preparation	Full-Stack Developer Guide	Python for JavaScript Devs

Mobile App

Download on the App Store

GET IT ON Google Play

Our Charity